Cahier des Charges — Rapport Final Projet Python ING3

Nov 7, 2026 · @mamadou

Projet : Engine de traitement et d'analyse de flux de données sécurisé en temps réel
Équipe : Mniangadou (Dev A) · Marlon (Dev B) · Baptiste (Dev C)
Livrable couvert : rapport PDF final (en remplacement du notebook initialement prévu, nouvelle consigne du prof)

1. Objectif et destinataire du rapport

Le rapport **remplace le notebook** initialement prévu dans l'énoncé. Nouvelle consigne du prof confirmée le 2026-11-07.

Le lecteur unique est le prof. Il doit pouvoir, en une lecture :

- Comprendre **ce que fait** votre pipeline (en 1 min via l'intro et le schéma d'architecture).
- Vérifier **que chaque point du barème est traité** (via un mapping explicite § 6 de ce cahier).
- Juger la **qualité du raisonnement et de l'implémentation** (via les sections techniques des modules).
- Lire une **preuve chiffrée** que le pipeline tient la charge (ratio 301 106× déjà mesuré).

Ton attendu :

- Technique, précis, affirmatif (jamais de « nous pensons que », « il semblerait que »).
- Phrases courtes, paragraphes courts.
- Pas de bla-bla d'introduction générique sur Python ou la cybersécurité — on va droit au but.
- Code toujours commenté, figures toujours numérotées et légendées.

Différence avec un notebook / un README :

	Notebook	README	Rapport
Lecteur	Dev qui exécute	Dev qui installe	Prof qui note
Code exécutable	Oui	Non	Non (illustratif)
Narration	Entre cellules	Minimale	Centrale
Figures	Rendues en live	Rares	Captures + schemas

	Notebook	README	Rapport
But	Prototype + exploration	Prendre en main	Convaincre et prouver

2. Format et contraintes matérielles

Paramètre	Valeur attendue
Format de rendu	**PDF** (export depuis Word / Google Docs / LaTeX)
Longueur cible	**15 à 25 pages** hors annexes (à confirmer avec le prof)
Police corps de texte	Calibri 11 ou Times New Roman 12, interligne 1,15
Police titres	Calibri 14-18 bold, hiérarchie claire H1/H2/H3
Police code	Consolas / Courier New 10, fond gris clair
Marges	2,5 cm partout
Numérotation pages	En pied de page, format `page X / N`
Numérotation figures	**Figure 1 — titre descriptif** sous chaque image
Numérotation tableaux	**Tableau 1 — titre descriptif** au-dessus
Code dans le texte	Blocs court (< 20 lignes). Code long en annexe
Annexes	Paginées A, B, C… Code source complet non répété (il est sur Git)

Règle d'or sur le code dans le rapport : on **ne recopie pas** le code complet. On montre **uniquement** les fragments qui prouvent un point du barème (ex. la ligne qui fait `yield from`, la boucle avec `else`, la signature `*args, **kwargs`). Le reste est sur le dépôt Git mentionné en intro.

Règle d'or sur les figures : chaque figure doit **répondre à une question** posée dans le texte. Une figure non commentée ne sert à rien. Exemples utiles ici :

- Schema d'architecture du pipeline (boîtes + flèches).
- Capture du terminal montrant la sortie du bench mémoire.
- Courbe du ratio taille fichier / RAM (petit, moyen, 12 Go).
- Graphique en barres — répartition des niveaux INFO/WARNING/ERROR/CRITICAL dans le dataset.
- Timeline ou histogramme — détection des rafales brute-force.

3. Structure globale du rapport

Sections obligatoires, dans l'ordre :

1. **Page de garde** — titre du projet, noms des 3 étudiants (avec numéros d'étudiant si applicable), filière ING3, année scolaire, logo école si dispo, nom du prof, date.
2. **Sommaire** — généré automatiquement, avec pagination.
3. **Introduction** — 1 page max : contexte, problématique, choix techniques généraux (Python 3.10+, venv, CLI, générateurs), plan du rapport.
4. **Architecture générale du projet** — 1 à 2 pages : schéma global du pipeline, arborescence du dépôt, flux de données ingestion → filtrage → analyse → règles.
5. **Module 1 — Ingestion & Générateurs** (Dev A) — 2 à 3 pages : objectif, choix d'implémentation (`yield` / `yield from`), parser tolérant aux données sales, modélisation des logs (dict + tuples immuables).
6. **Module 2 — Pipeline & Programmation Fonctionnelle** (Dev B) — 2 à 3 pages : `filter` / `map` / `lambda`, normalisation, `match-case` sur les 3 formats.
7. **Module 3 — Agrégation & Traitement avancé** (Dev C) — 2 à 3 pages : `set` / `frozenset`, opérations d'ensembles, `defaultdict`, détection brute-force (`while` / `for` / `else`).
8. **Module 4 — Moteur de règles & Introspection** (Dev C) — 1 à 2 pages : `apply_security_rules(*args, **kwargs)`, audit `globals()` / `locals()`.
9. **Génération du dataset et nettoyage** (Dev A) — 1 à 2 pages : script `scripts/generate_logs.py`, statistiques du dataset 12 Go, 7 % de lignes sales injectées, variantes de saleté.
10. **Résultats et mesures** — 2 à 3 pages : bench mémoire (ratio 301 106×), stats de parsing sur le 12 Go, top IPs blacklistées actives, détection brute-force, exemples de sorties CLI.
11. **Difficultés rencontrées et choix techniques** — 1 page : ce qu'on a essayé et abandonné, ce qui a été corrigé en cours de route (ex. microsecondes dans timestamps, BLACKLIST désynchronisée, perf générateur).
12. **Répartition du travail** — 0,5 page : tableau clair qui a fait quoi (fichiers, commits, PRs).
13. **Conclusion** — 0,5 page : bilan vs objectifs, ouvertures (ex. streaming réseau réel, extension à plus de règles).
14. **Annexes** :
    - A. Lien vers le dépôt Git.
    - B. Contenu du `requirements.txt`.
    - C. Exemples de lignes de logs (propres + sales).
    - D. Sortie complète du bench mémoire.

- E. Commandes CLI utiles.
- F. Grille d'autoevaluation vs barème.

4. Contenu détaillé attendu dans chaque section

3. Introduction (Dev A)

- Phrase d'accroche : « Le projet consiste à concevoir un pipeline Python capable d'ingérer, nettoyer et analyser des logs volumineux (10–15 Go) sans saturer la RAM. »
- Contexte de la commande (cours ING3, trinôme, durée 4 semaines).
- 3 contraintes clés : Python 3.10+, environnement virtuel isolé, générateurs obligatoires.
- Annonce du plan (6 modules du rapport en une phrase).

4. Architecture générale (Dev A)

- Schéma pipeline en 1 figure : `scripts/generate_logs.py` → `.log` → Module 1 (lecture + parsing) → Module 2 (filtre + map + router) → Module 3 (agrégation + brute-force) → Module 4 (règles + audit) → sortie CLI.
- Arborescence du dépôt commentée (copie allégée du README).
- Justification des choix structurants : package Python, un module = une responsabilité, main.py comme chef d'orchestre.

5. Module 1 — Ingestion (Dev A)

- Rappel du besoin : lire 12 Go sans RAM.
- Explication de `yield` vs lecture `.read()` complète (schéma ou analogie).
- Extrait de `log_generator.py` : les 10 lignes de `read_log_file` + `read_multiple_sources` (montrer `yield` et `yield from`).
- Modélisation : `LogEntry` (TypedDict), `Severity` (Enum), `ConnectionMeta` (tuple immuable).
- Parser : stratégie de rejet des lignes sales (retour `None` + `filter(None, ...)` côté pipeline).
- Résultat : 16 tests pytest, dont tolérance microsecondes, IP invalides, niveaux inconnus, user manquant.

6. Module 2 — Pipeline fonctionnel (Dev B)

- Enjeu : chaîner filtrage + transformation sans boucles impératives.
- Extrait de `filters.py` : `filter(None, ...)` et `filter(lambda log: ..., ...)`.

- Extrait de `transforms.py` : `map(normalize, logs)`.
- Extrait de `router.py` : les 3 `case` du `match-case` (string / dict / tuple de connexion) avec exemples d'entrée.
- Justification du choix fonctionnel (préservation de l'itérateur, pas d'allocation intermédiaire).

7. Module 3 — Agrégation (Dev C)

- Extrait de `counters.py` : `defaultdict(int)` avec exemple de résultat.
- Extrait de `sets_ops.py` : les 4 opérateurs `& | - ^` entre les IPs actives et la `BLACKLIST` (frozenset).
- Extrait de `bruteforce.py` : **la boucle `while` / `for` avec clause `else`** (point critique barème, encadrer en couleur).
- Explication du critère de détection (N échecs consécutifs, reset si succès).

8. Module 4 — Rule engine & introspection (Dev C)

- Extrait de `rule_engine.py` : la signature `apply_security_rules(*args, **kwargs)` (encadrer).
- Exemple d'appel avec 2 règles passées en arguments.
- Extrait de `audit.py` : usage de `globals()` ET `locals()` dans la même fonction.
- Discussion : pourquoi les fonctions sont traitées comme des objets de première classe.

9. Génération du dataset (Dev A)

- Rappel consigne prof : 10–15 Go + variété + données sales.
- Stats du fichier généré : 12,11 Go, 119 510 991 lignes, 7,0 % sales, 2 389 rafales brute-force.
- Les 13 variantes de lignes sales (tableau).
- Justification du `--seed 42` (reproductibilité équipe).

10. Résultats et mesures (Dev A + Dev C)

- **Figure principale (Dev A)** : tableau ou courbe `taille fichier (5 Mo, 2,75 Go, 12 Go) → RAM pic`. Reprise des chiffres de BENCH.md.
- Bench mémoire : ratio **301 106×** sur 12 Go, 42 Ko de RAM. Phrase de verdict : « le pipeline tient une taille arbitraire d'entrée ».
- **Figure (Dev C)** : répartition des niveaux de sévérité (bar chart).
- **Figure (Dev C)** : top 10 des IPs avec le plus d'erreurs.

- **Figure (Dev C)** : nombre de rafales brute-force détectées + exemples de séries interrompues.
- Capture d'écran de la sortie CLI complète du `main.py`.

11. Difficultés rencontrées (tous)

- Bug parser sur timestamps avec microsecondes (détecté par le premier run, fix regex).
- Optimisation du générateur (de 5 Mo/s à 12,6 Mo/s via buffering 4 Mo + writelines).
- BLACKLIST désynchronisée entre générateur et analytics (corrigé en review).
- Chaque dev raconte **un** choix technique qu'il a dû trancher.

12. Répartition du travail (tous)

- Tableau simple : qui a écrit quels fichiers, nombre de commits par personne, PRs ouvertes / relues.

13. Conclusion (Dev A)

- Bilan vs objectifs : tous les points du barème couverts (voir § 6).
- Ouvertures : streaming réseau réel (Kafka, sockets), plus de règles de sécurité configurables, visualisation temps réel.
- Une phrase de clôture, sans emphase excessive.

5. Répartition de la rédaction

Principe : chacun écrit les sections techniques de SA partie (il la connaît le mieux), et un rédacteur unique coordonne les sections transversales (intro, architecture, conclusion) pour garder un ton homogène.

Section	Rédacteur principal	Relecteur
1. Page de garde	Dev A	Tous
2. Sommaire	Généré auto	—
3. Introduction	**Dev A**	Dev B + Dev C
4. Architecture générale	**Dev A**	Dev B + Dev C
5. Module 1 — Ingestion	**Dev A**	Dev B

Section	Rédacteur principal	Relecteur
6. Module 2 — Pipeline	**Dev B**	Dev A
7. Module 3 — Agrégation	**Dev C**	Dev A
8. Module 4 — Rule engine	**Dev C**	Dev A
9. Génération du dataset	**Dev A**	Dev C
10. Résultats et mesures	**Dev A** (partie bench) + **Dev C** (partie analyses)	Dev B
11. Difficultés rencontrées	**Tous** (un paragraphe chacun)	—
12. Répartition du travail	Dev A (compile)	Tous valident
13. Conclusion	**Dev A**	Dev B + Dev C
Annexes	Dev A (compile depuis les fichiers du repo)	Tous

Règles de collaboration :

- Un seul document partagé (Google Docs recommandé pour l'édition parallèle, export PDF final).
- Chaque rédacteur signe chaque paragraphe avec ses initiales en commentaire tant que le doc n'est pas figé (facilite la review, retiré à la fin).
- Les relecteurs commentent dans le doc, le rédacteur accepte / discute.
- **Dev A est responsable de la cohérence globale** (ton, numérotation figures/tableaux, mise en forme finale, export PDF).

6. Correspondance barème → sections du rapport

Chaque critère du barème doit être prouvé explicitement dans le rapport, pas seulement implicite dans le code. La table suivante est aussi à reproduire en **annexe F** (grille d'autoévaluation).

Critère (points)	Section du rapport	Preuve concrète à inclure
Environnement & Livrables (2 pts)	§ 4 Architecture + Annexe A/B	Lien Git + commande `pip install -r requirements.txt` qui marche + distinction `.py` / absence de notebook

Critère (points)	Section du rapport	Preuve concrète à inclure
Gestion mémoire & Générateurs (4 pts)	§ 5 Module 1 + § 10 Résultats	Extrait code `yield` + `yield from` + **tableau ratio taille/RAM** avec mesure sur 12 Go (301 106×)
Structures de données complexes (4 pts)	§ 5 + § 6 + § 7	Extraits : `tuple ConnectionMeta`, `set/frozenset BLACKLIST` + 4 opérateurs, `defaultdict`, `match-case` à 3 cas
Programmation fonctionnelle (4 pts)	§ 6 Module 2 + § 8 Module 4	Extraits : `map()`, `filter()`, `lambda`, passage de fonctions en argument dans `apply_security_rules`
Moteur de règles & Introspection (3 pts)	§ 8 Module 4	Signature `apply_security_rules(*args, **kwargs)` + exemple d'appel + fonction `audit_runtime_state` avec `globals()` ET `locals()`
Qualité du code & Robustesse (3 pts)	§ 7 Module 3 + § 5 Module 1 + § 11 Difficultés	**Boucle `while` / `for` avec clause `else`** (encadrée), typage, docstrings, gestion d'exceptions explicite, parser tolérant

Règle d'or : pour chaque ligne du tableau ci-dessus, le lecteur doit pouvoir **pointer un paragraphe précis** du rapport et dire « oui c'est traité ». Si vous n'arrivez pas à faire ce pointage pour un critère, il manque un paragraphe.

Astuce de présentation : ajoutez des **encadrés de couleur** (vert clair) autour des 5 éléments critiques :

- `yield` / `yield from` (p.5)
- `frozenset` + 4 opérateurs d'ensemble (p.7)
- `match-case` (p.6)
- Clause `else` d'une boucle (p.7)
- `*args, **kwargs` + `globals()` / `locals()` (p.8)

Le prof ne lira pas chaque ligne — ces encadrés lui donnent le signal visuel « tout est là ».

7. Planning de rédaction (dernière semaine)

L'idée : **le rapport se rédige en parallèle du développement**, pas après. Chaque fois qu'un module est fini et PR mergée, son auteur rédige la section correspondante **tant qu'il a le code en tête**.

Semaine finale (J1 = lundi) :

J1 — Plan et rassemblement

- Visio 1h30 à 3 : validation du plan détaillé de ce cahier des charges, création du Google Doc partagé, injection du squelette (§ 8 ci-dessous).
- Dev A : commence l'intro et l'architecture (sections 3-4).
- Dev B : commence Module 2 (section 6).
- Dev C : commence Module 3 et 4 (sections 7-8).
- Fin de journée : squelette complet avec les titres de toutes les sections, même vides.

J2 — Rédaction gros bloc

- Chacun termine sa ou ses sections techniques (5 à 8 pages par personne).
- Dev A commence à générer les captures d'écran et figures (sortie bench, arborescence, etc.).
- Fin de journée : 80 % du texte est en place.

J3 — Résultats, figures, section 10

- Dev A : termine la section Résultats avec bench mémoire (tableau + verdict).
- Dev C : génère les graphes (matplotlib) pour la répartition sévérité, top IPs, brute-force. Les exporte en PNG et les insère.
- Dev B : tableau de répartition du travail, section Difficultés rencontrées.
- Fin de journée : rapport complet, pas encore relu.

J4 — Relecture croisée + corrections

- Chacun relit la section d'un autre (Dev A relit Dev C, Dev B relit Dev A, Dev C relit Dev B) et commente dans le doc.
- Corrections.
- Vérification des encadrés verts barème (les 5 points clés du § 6).
- Remplissage des annexes.

J5 — Mise en forme finale

- Dev A : passe sur le document complet, uniformise les polices, numérote figures et tableaux, génère le sommaire automatique, soigne la page de garde.
- Vérification des liens (lien dépôt Git fonctionnel, précisément https://github.com/Mamadouedu/projetspython).
- Export PDF test.

J6 — Relecture finale (buffer)

- Dernière passe : fautes de frappe, cohérence des chiffres (un chiffre comme 12 Go doit être partout identique).
- Vérification que le rapport se tient seul (un lecteur qui n'a pas le code doit tout comprendre).

J7 — Rendu

- Export PDF final.
- Vérification taille < 10 Mo (si plus, compresser les images).
- Dépôt selon modalité du prof (mail, plateforme, impression…).
- **Tag Git** `v1.0.0` sur la version du code qui correspond au rapport.

Buffer de sécurité : le rendu est prévu J7 mais tout doit être jouable **J5 en fin de journée**. J6-J7 ne sont que du buffer pour les imprévus.

8. Modèle de squelette + conseils rédactionnels

Squelette à copier-coller dans Word / Google Docs

```
[Page de garde]
   Logo école
   PROJET FINAL ING3 — PYTHON
   Engine de traitement et d'analyse de flux de données
   sécurisé en temps réel
   Mniangadou · Marlon · Baptiste
   Filière ING3 — Année 2026-2027
   Prof : [nom]
   Date de rendu : [date]

[Sommaire — généré auto]

1. Introduction
   1.1 Contexte
```

1.2 Problématique
1.3 Choix techniques généraux
1.4 Plan du rapport

2. Architecture générale
2.1 Vue d'ensemble du pipeline [Figure 1]
2.2 Arborescence du dépôt
2.3 Justification des choix structurants

3. Module 1 — Ingestion & Générateurs mémoire
3.1 Besoin et contraintes
3.2 Implémentation yield / yield from [Extrait code 1]
3.3 Modélisation : LogEntry, Severity, ConnectionMeta
3.4 Parser tolérant aux données sales
3.5 Tests unitaires

4. Module 2 — Pipeline & Programmation fonctionnelle
4.1 Enjeu : itérateur préservé bout en bout
4.2 Filtrage avec filter() + lambda [Extrait code 2]
4.3 Transformation avec map() [Extrait code 3]
4.4 Routage avec match-case [Extrait code 4]

5. Module 3 — Agrégation & Traitement avancé
5.1 Comptage avec defaultdict [Extrait code 5]
5.2 Opérations d'ensembles (frozenset, &, |, -, ^) [Extrait code 6]
5.3 Détection brute-force (while/for + else) [Extrait code 7, encadré]

6. Module 4 — Moteur de règles & Introspection
6.1 apply_security_rules(*args, **kwargs) [Extrait code 8]
6.2 Audit via globals() et locals() [Extrait code 9]
6.3 Fonctions comme objets de première classe

7. Génération du dataset et nettoyage
7.1 Contraintes prof : 10-15 Go, variete, donnees sales
7.2 Statistiques du fichier généré [Tableau 1]
7.3 Les 13 variantes de lignes sales [Tableau 2]

8. Résultats et mesures
8.1 Bench mémoire : ratio 301 106x [Tableau 3, Figure 2]
8.2 Répartition des niveaux de sévérité [Figure 3]
8.3 Top 10 IPs avec erreurs [Figure 4]
8.4 Détection brute-force [Figure 5]
8.5 Sortie CLI complète [Capture 1]

9. Difficultés rencontrées et choix techniques
9.1 Bug timestamps microsecondes

```
   9.2 Optimisation du générateur
   9.3 Synchronisation de la BLACKLIST

10. Répartition du travail [Tableau 4]

11. Conclusion
    11.1 Bilan vs objectifs
    11.2 Ouvertures

Annexes
   A. Dépôt Git
   B. requirements.txt
   C. Exemples de lignes de logs
   D. Sortie complète du bench mémoire
   E. Commandes CLI
   F. Grille d'autoévaluation vs barème
```

Conseils rédactionnels

Style :

- Phrases courtes (15-20 mots max).
- Un paragraphe = une idée.
- Affirmer plutôt que suggerer : « Le ratio atteint 301 106× » (bon) ; « On pense que le ratio pourrait être intéressant » (mauvais).
- Jamais de « I » ou « we » en français : utiliser « nous » ou tournure impersonnelle (« le pipeline lit »).

Figures et tableaux :

- Chaque figure appelée explicitement dans le texte : « La figure 2 montre que la RAM reste plate… ».
- Légende sous la figure, titre au-dessus du tableau.
- Capture d'écran du terminal : fond sombre OK mais lisibilité avant tout ; couper les parties vides.

Extraits de code :

- 5 à 15 lignes max par extrait.
- Toujours une ligne de commentaire au-dessus : « Extrait 7 — La boucle `while` contient la clause `else` qui valide la réussite d'un cycle complet de détection. »
- Si l'extrait dépasse, synthétiser avec `# ...` pour montrer juste la partie intéressante.

Pièges à éviter :

- ❌ Recopier tout le code du projet dans le rapport (c'est sur Git, lien dans l'annexe A).

- ❌ Figures sans légende ni appel dans le texte.
- ❌ Chiffres incohérents (12 Go vs 12,1 Go vs 12402,59 Mo → choisir un format et s'y tenir).
- ❌ Sections « vides » style « Ce module traite les logs » sans montrer concrètement.
- ❌ Oublier la section Difficultés (le prof veut voir que vous avez réfléchi, pas juste codé).
- ❌ Conclusion épique à rallonge (max 1 page).
- ❌ Fautes d'orthographe (faire relire par quelqu'un hors équipe si possible).